**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# LESSON 256 · GPUs, multicore and parallel systems

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. Why doesn't a smaller assembly instruction count guarantee a faster program?

Q2. What is a core?

Q3. Two independent tasks each take 4 minutes; what is ideal concurrent time?

Q4. Given int[] a={1,2,3,4}, perform a[i]=a[i]*2 for even indexes 0 and 2 only. Give final array and its sum.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

GPU — Processor designed for highly parallel workloads.

multicore — CPU containing multiple execution cores.

data parallelism — Apply the same operation to many independent data elements.

bandwidth — Volume of data transferred in a given time.

synchronisation — Coordinating shared work and results.

## New learning

GPUs and data parallelism — A GPU contains many execution resources suited to repeating similar calculations across graphics pixels or machine-learning tensors.

Multicore CPUs — Multiple CPU cores can run independent threads/processes, enabling greater throughput when tasks can be parallelised.

Evaluate overhead — Data transfers, branching, synchronisation and sequential tasks can reduce the ideal speedup. A GPU is not automatically best for every problem.

## Worked example

Compare 12 independent image tiles split across four cores under an ideal equal-work model.

1. Predict first — Divide 12 independent equal-sized tiles among four available cores.

2. Explain and check — Each core processes three tiles, so ideal completion is around one quarter of serial time only if work and overhead allow.

# We Do · Practise with guidance

An encryption batch has 20 independent blocks and five worker cores. How many blocks per core ideally?

```
20 blocks / 5 workers = 4 blocks each
```

Predict first

The blocks can be divided evenly into five independent groups.

Explain and check

Each core receives four blocks; coordination overhead and data movement may affect real runtime.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Give two uses of GPUs, including one not based on graphics, and explain a common characteristic of both.

### Question 2

A task has 24 independent identical chunks and six CPU cores. Calculate ideal work per core and describe two constraints on real speedup.

### Question 3

A video-editing workflow must first decode frames sequentially, then apply the same filter independently to each frame, before assembling output. Explain which parts suit GPU execution and why.

# Exit Ticket · Answer before checking

GPUs always execute any sequential algorithm faster than a CPU.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. Different instructions have different costs, and decoding, caches and pipelining affect performance.

Q2. An execution unit capable of carrying out its own instruction stream.

Q3. 4 minutes, ignoring overhead.

Q4. Final [2,2,6,4], sum=14. Indexes 0 and 2 were changed, not the even-valued elements.

### You Do pages · Independent questions

Q1. [4] Graphics/rasterisation [1]; large-scale matrix operations/AI [1]; many independent similar calculations [2].

Q2. [6] 24/6=4 chunks per core [2]; serial work limits speedup [2]; synchronisation, memory-bandwidth or scheduling overhead [2].

Q3. [6] Independent per-frame filter calculations can be parallel [2]; GPU data-parallel execution benefits them [1]; decoding dependencies may constrain speedup [1]; assembly waits for processed frames [1]; host–GPU transfer and memory bandwidth reduce ideal gains [1].

### Exit Ticket · Model answer

False. Highly dependent tasks may not benefit and transfer/launch overhead can dominate.

# Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

**1. Copy the whole original question:**

**2. Write a complete new answer in GREEN PEN. Show working if needed:**

**BLACK PEN** Answer all questions first.

**GREEN PEN** Then: tick correct answers; rewrite mistakes.

**Do Now** Recall previous learning | **I Do** Read new knowledge | **We Do** Practise with guidance | **You Do** Work independently | **Review & Improve** Check my understanding